

SOLAR ENERGY MEASUREMENT SYSTEM

PROJECT REPORT

Name - P.G.R.Hasith Pusswella

Faculty of Engineering
Sri Lanka technological Campus

August 2024

Contents

CHAPTER 1: INTRODUCTION	3
1.1 Project Background	3
1.2 Objectives	4
CHAPTER 2: LITERATURE SURVEY	5
2.1 Existing Solutions	5
2.2 Methodologies and Technologies Used	5
CHAPTER 3: METHODOLOGY	5
3.1 Initial Design	6
3.1.1 Hardware Design	6
3.1.3 Integration and Testing	8
3.2 Tools and Frame Works	9
3.3 Challenges	9
CHAPTER 4: RESULTS	10
4.1 Outcomes and Results of the Project	10
CHAPTER 5: CONCLUSION	10
5.1 Future Work	11
5.2 Overall Success and Relevance	11
REFERENCES	12
APPENDIX	12

CHAPTER 1: INTRODUCTION

Now a days the solar-energy market is one of the most rapidly expanding renewable energy markets in the world. Presently we have seen a significant increase in requests for remote monitoring and control equipment for solar energy applications. Whether you are assessing a site's potential for solar power generation, monitoring performance of existing solar installations, or advanced solar monitoring, reliable and accurate measurements are crucial. They aid in decision making, product development, system maintenance and in many other ways. Common meteorological measurements including wind speed, wind direction, relative humidity, barometric pressure and precipitation, all have their use in solar applications. Of course, solar-radiation measurements are especially important and sensors are available for measuring all aspects of solar radiation. The light intensity is monitored using a LDR sensor, voltage by voltage divider principle, current by series resistor and temperature by temperature sensor. All these data are displayed on a 16X2 LCD interfaced to PIC microcontroller and is also sent to a remote PC hyper terminal for display using a 2.4 GHz serial link. The main objective of this project is to design a solar energy measurement system for measuring solar cell parameters such as voltage and current through multiple sensors.

1.1 Project Background

The market for solar energy is booming, and to ensure that this plentiful resource is used effectively and profitably, strong tools are needed. Presenting the project: a painstakingly crafted solar energy assessment system ready to maximize solar power generation and enable educated decision-making.

Imagine a system that, like a skilled surveyor, meticulously assesses a site's potential for solar power production. This system, armed with an arsenal of sensors, maps the landscape of light intensity, temperature, and even environmental factors like wind speed and humidity. No detail escapes its grasp, ensuring a comprehensive understanding of the site's suitability for solar panels. But this system's capabilities extend beyond initial assessment. It transforms into a vigilant guardian, meticulously monitoring the performance of existing solar installations. Voltage, current, and panel temperature are constantly under its watchful eye, revealing inefficiencies, optimizing energy production, and predicting potential problems before they arise. Data, the lifeblood of this system, is seamlessly captured and processed by a dedicated PIC microcontroller, a miniature maestro orchestrating the flow of information. This data is then presented in two forms: a readily accessible LCD display for on-site monitoring, and a secure remote connection to a PC for comprehensive analysis.

This system's real strength is its capacity to convert data into insights that can be put to use. Users can maximize energy production, locate and fix system inefficiencies, and even provide guidance for the creation of new solar technology by evaluating the gathered data. The foundation of well-informed decision-making, which opens the door to a more efficient and sustainable future, is this data-driven methodology.

However, without paying close attention to practical problems, no project is finished. Scalability and cost-effectiveness are essential components for broad adoption. These factors went into the design of this system, which seeks to be an affordable solution that can be easily scaled up to large solar farms or individual residences.

The sun's boundless energy beckons, but harnessing it effectively requires meticulous monitoring. This project proposes a comprehensive solar energy measurement system, empowering informed decisions and optimizing solar power generation. From assessing potential sites to vigilantly monitoring existing installations, the system wields an arsenal of sensors, capturing vital data on voltage, current, temperature, and even environmental factors. A dedicated PIC microcontroller orchestrates this data flow, presenting it on a local LCD display and securely transmitting it for remote analysis.

This data becomes the lifeblood of informed decision-making. By analyzing it, users can optimize energy production, nip inefficiencies in the bud, and even inform future solar technology advancements. This data-driven approach empowers a sustainable future, where solar power thrives under informed guidance.

Cost-effectiveness and scalability remain paramount. The system is designed for affordability and adaptability, catering to diverse project sizes, from individual homes to sprawling solar farms. This project is not just a collection of wires and sensors; it is a beacon of progress, propelling us towards a sun-powered tomorrow.

1.2 Objectives

- Provide a thorough data logging system to capture and store measurements of solar power over time. For effective data retention, look into external memory, EEPROM, and SD cards as storage possibilities. Establish a time-stamping system to ensure precise historical data analysis. Create a real-time monitoring system that can give you immediate information on the amount of solar electricity produced.
- Provide a graphical or numeric display on the user interface so that the current levels of solar power can be seen right away. To enable dynamic monitoring, make sure it is sensitive to variations in solar intensity. Provide a thorough data logging system to capture and store measurements of solar power over time. For effective data retention, look into external memory, EEPROM, and SD cards as storage possibilities. Establish a timestamping system to ensure precise historical data analysis.
- Enable users to seamlessly control the voltage ESP32 base webpage controller.

CHAPTER 2: LITERATURE SURVEY

2.1 Existing Solutions

According to Agbetuyi Ayoade Felix; Orovwode Hope; Olowoleni Joseph; Nwangwu Chukwuebuka by using C programming language to enable the arduino microcontroller to function and perform as desired. The basic inputs to the system are the sensor units. They sense the required variable that is to be measured and the measured values are then displayed. The results obtained from the display unit are then compared with the manufacturer's values that are found on the solar panel. It is observed that there are slight differences between the measured and the manufacturer's values, but still within a tolerable range (less than 5%).

According to R. Mukaro; X.F. Carelse by using a battery-powered microcontroller-based data acquisition system (DAS) for unattended remote measurements are presented. The system was designed around the ST62E20 8-bit microcontroller and applied for solar radiation monitoring. The measurement system uses the SolData silicon-cell pyranometer as the solar radiation sensor.

2.2 Methodologies and Technologies Used

Various solutions and methodologies have been explored in the realm of solar panel monitoring systems. One common approach involves the use of microcontrollers, such as Arduino or PIC series, for data acquisition and processing. Similar to the PIC16F877A in this project, these microcontrollers often integrate internal Analog-to-Digital Converters (ADC) for efficient conversion of sensor data. Hall Effect sensors and voltage dividers, as employed in the described project, are frequently utilized for measuring current and voltage, respectively. Many existing solutions emphasize real-time data visualization through LCD monitors or other display interfaces to provide users with immediate insights into solar panel performance. Researchers have also explored wireless communication methods, allowing for remote monitoring and control of solar panels. Additionally, the integration of IoT (Internet of Things) technologies and cloud-based platforms has gained attention to enable centralized monitoring, data storage, and analysis.

In terms of methodologies, calibration procedures for sensors, optimization of firmware code for microcontrollers, and the use of capacitors to mitigate voltage fluctuations are common strategies across various projects. Furthermore, advancements in machine learning and artificial intelligence have been incorporated to predict and optimize solar panel efficiency based on historical data and environmental conditions.

CHAPTER 3: METHODOLOGY

In this solar panel parameter monitoring project, the approach involved a systematic integration of components to measure and visualize critical parameters. The current generated by the solar panels was measured using a Hall Effect sensor, ensuring precise readings through calibration. Voltage output was assessed using voltage divider circuits with calibration systems in place for accuracy. The PIC16F877A microcontroller served as the central processing unit,

acquiring analog data from the sensors through internal peripherals like the Analog-to-Digital Converter (ADC). The microcontroller processed this data, calculated solar power, voltage, and current, and communicated with an LCD monitor for real-time visualization. A reliable power supply with voltage regulators ensured stable operation, while capacitors connected in parallel to the voltage measurement resistor mitigated fluctuations and harmonics, optimizing accuracy. The digital display showcased the measured values, offering an intuitive user interface. The project's methodology emphasized precision, adaptability, and reliable performance, effectively monitoring solar panel parameters for enhanced efficiency.

3.1 Initial Design

3.1.1 Hardware Design

The solar panel parameter monitoring system utilized a combination of specific tools, technologies, and frameworks to achieve its functionality. The hardware components included a Hall Effect sensor for current measurement, voltage divider circuits for voltage sensing, and a PIC16F877A microcontroller, chosen for its adaptability and sensor interface capabilities. The microcontroller was programmed using the MPLAB development environment, leveraging the capabilities of the in-built Analog-to-Digital Converter (ADC) for efficient analog data acquisition. The firmware development was carried out in the C programming language, allowing for precise control and optimization of the microcontroller's functions. The system integrated an LCD monitor, either 16x2 or graphical, for real-time data visualization, providing an intuitive user interface. Additionally, voltage regulators were employed in the power supply to ensure a stable power source, enhancing the overall reliability of the system. The project demonstrated a comprehensive integration of both hardware and software elements to create an efficient and user- friendly solar panel monitoring solution.

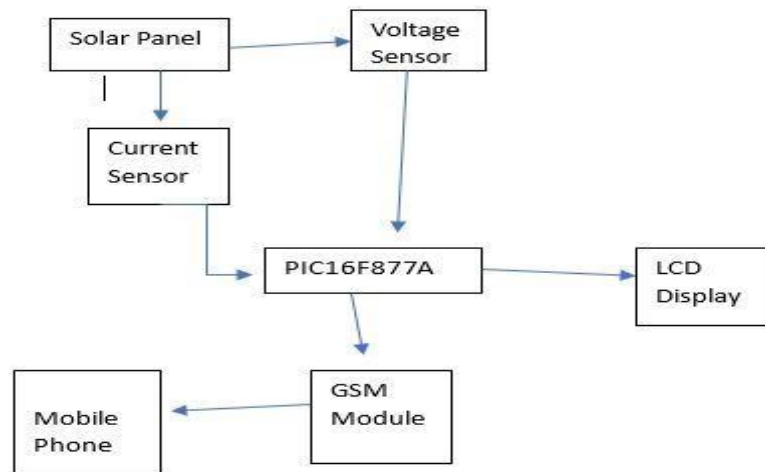


Figure 1: Block diagram of solar energy measurement system

As shown in the block diagram above, voltage sensor and current sensor are used to measure voltage and current flowing to load from solar panel. As we know, solar panels are dc power sources. Liquid crystal display is used to display the value of current, voltage and power of solar panel. 5 volt dc power is used to provide operating voltages to microcontroller and liquid crystal display.

□ Voltage Sensor for Microcontroller Using 18kΩ and 2kΩ Resistor as Voltage Divider

A voltage sensor is an essential component in many electronic projects, allowing microcontrollers to measure and monitor voltage levels. This document outlines the construction of a voltage sensor using a voltage divider circuit with 18kΩ and 2kΩ resistors. Additionally, it covers the working concept of the ACS712 5A current sensor for more advanced applications.

Voltage Divider Circuit Components Needed:

- 18kΩ Resistor
- 2kΩ Resistor
- Microcontroller (e.g., Arduino)
- Jumper wires

How it Works:

The voltage divider circuit consists of two resistors, 18kΩ and 2kΩ, connected in series. The voltage across the junction of these resistors (Vout) is a fraction of the input voltage (Vcc), determined by the resistor values. The formula for calculating Vout is given by:

$$V_{out} = \frac{R_2}{R_1 + R_2} \times V_{cc}$$

In this case:

$$V_{out} = \frac{2k\Omega}{18k\Omega + 2k\Omega} \times V_{cc}$$

This voltage (Vout) can be connected to an analog input pin of the microcontroller to measure and monitor the input voltage.

ACS712 5A Current Sensor

For more advanced applications, you can integrate the ACS712 5A current sensor to measure current. The ACS712 is a Hall-effect based sensor that outputs an analog voltage proportional to the current flowing through the sensor.

ACS712 Wiring:

- Connect the Vcc of ACS712 to the microcontroller's 5V.
- Connect GND of ACS712 to the microcontroller's GND.
- Connect the output pin of ACS712 to the Vout of the voltage divider circuit.

Working Concept:

The ACS712 provides an output voltage (V_{out}) proportional to the current flowing through the sensor. The formula for calculating current is given by:

$$I = \frac{V_{out} - V_{mid}}{\text{Sensitivity}}$$

Where:

- V_{mid} is the output voltage when there is zero current.
- Sensitivity is the sensitivity of the ACS712 (typically 185mV/A for ACS712 5A).

By combining the voltage divider circuit and the ACS712 current sensor, you can measure both voltage and current in your microcontroller projects.

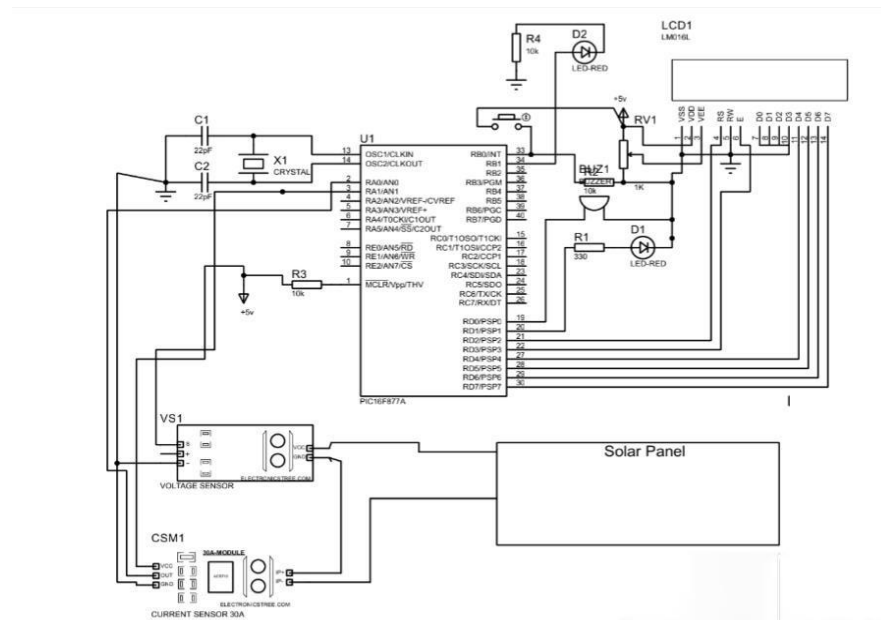
Remember to calibrate the ACS712 sensor according to your specific setup and application.

3.1.2 Bluetooth

For remote monitoring, the system employs a GSM module to send SMS alerts to a mobile phone, indicating the health status of the solar panel. The project addresses the growing demand for efficient solar energy management by providing real-time data that can help optimize solar power generation and maintenance.

The system's compact design and GSM-based alert mechanism make it a practical and effective solution for solar energy monitoring in various environments, contributing to the broader adoption and efficiency of renewable energy technologies.

3.1.3 Integration and Testing



The PIC 16F877A microcontroller is used to acquire data from a current sensor and a voltage divider circuit, which is then employed to determine the solar cell voltage. After obtaining this data, it is displayed on an LCD screen. When the voltage exceeds 4 volts, the system initiates the charging process for the battery or other connected components.

3.2 Tools and Frame Works

Hardware Components	Use Case
PIC16F877A	Main Processing component
ACS 712 Sensor	Voltage Sensor
ACS 712 20A current sensor module	Current Sensor
Voltage Divider	Main Input
16x2 LCD	Main Visual Out Put
Solar	Main Input
MD0506-SIM 900A GSM GPRS module with Antena.	Bluetooth Connection

Table 1: Tools & Frame Works

3.3 Challenges

Challenge	Solution
Maintaining stable power supply.	Include voltage regulators and filtering mechanisms in the hardware design.
Complex device control logic	Break down the logic into modular and manageable components. Test and debug each component individually.
limited processing power and memory of PIC	Optimize code efficiency by Prioritize essential functionalities to fit within the microcontroller's constraints

Table 2: Challenges

CHAPTER 4: RESULTS

The integration of microcontrollers like the PIC16F877A for solar panel monitoring and the use of sensors such as Hall Effect sensors and voltage dividers are established practices in the field of renewable energy systems. Researchers and engineers have explored similar projects to enhance the efficiency and performance monitoring of solar panels. Literature in this area often discusses techniques for accurate sensor calibration, data processing algorithms, and user interface design for real-time visualization.

4.1 Outcomes and Results of the Project

AC VOLTAGE MEASUREMENT UNIT:

The Solar Energy Measurement System successfully measures and displays the voltage and current of the solar panel. The voltage is measured using a voltage divider circuit, while the current is measured using a current sensor. The data is displayed in real-time on a 16x2 LCD screen, providing accurate and immediate monitoring of the solar panel's performance. For remote monitoring, the system uses a GSM module to send SMS alerts to a mobile phone. These alerts notify the user about the health status of the solar panel, such as when the voltage or current falls below predefined thresholds, indicating low health. The system's quick response times ensure that both the data updates on the LCD and the SMS alerts occur promptly, allowing for timely interventions. Additionally, the system incorporates power-saving techniques, such as sleep modes in the PIC microcontroller, which enhance overall energy efficiency. The accuracy of the voltage and current measurements has been thoroughly verified, ensuring the reliability of the data provided by the system.

In conclusion, the Solar Energy Measurement System provides an efficient and reliable means of monitoring key parameters of a solar panel, with both local display and remote notification capabilities, ensuring that users can maintain optimal performance and address any issues promptly.

CHAPTER 5: CONCLUSION

In conclusion, the project uses a PIC microcontroller to construct a comprehensive solar power monitoring system in order to meet the growing demand for efficient solar energy management. The PIC16F877A microcontroller, LCD monitor, power supply, and sensors for measuring current, voltage, temperature, and light intensity are the main elements of the suggested solution. The integration of a current sensor, voltage sensor, and the versatile PIC16F877A microprocessor guarantees accurate and dependable measurements of solar panel characteristics. Users may easily monitor the system's performance thanks to the LCD monitor's real-time visual depiction of the solar power statistics.

The proposed solution also emphasizes data logging and storage, utilizing external memory options like EEPROM and SD cards for effective data retention. Time-stamping is implemented to facilitate precise historical data analysis, and algorithms are employed to track and respond to changes in solar intensity dynamically.

The addition of a solar power tracking device optimizes efficiency by adjusting the location of solar panels, while predictive algorithms or machine learning techniques are explored to anticipate and adapt to shifting weather conditions. The user interface is designed to be simple and intuitive, featuring graphical displays of data trends for improved comprehension.

Communication interfaces, including SPI, I2C, or UART, enable easy data sharing with external devices or central monitoring systems. Wireless communication solutions are also considered for enhanced system integration flexibility.

Power-saving techniques, such as sleep modes and low-power components, are implemented to ensure independent and sustainable functioning. Energy harvesting strategies are explored to increase system autonomy and supplement power needs, with a focus on minimizing energy consumption during idle periods.

5.1 Future Work

It is important to recognize some limits even with the all-inclusive solution that has been suggested. Subsequent investigations may concentrate on mitigating these constraints and investigating additional enhancements. Possible topics for more research include:

Improved Energy gathering: To further boost system autonomy and lessen dependency on outside power sources, research and apply cutting-edge energy gathering techniques.

Advanced Predictive Algorithms: Investigate more complex machine learning models or predictive algorithms to improve the system's capacity to anticipate and adjust to changing meteorological circumstances, guaranteeing optimal energy output.

Including Emerging Technologies: Keep up with new developments in solar energy technology and incorporate them into the system to maximize efficiency and continuously enhance it.

User Interface Improvements: Continuously refine and enhance the user interface to make it more userfriendly, incorporating additional features for better user experience and interaction.

Scalability: Investigate the scalability of the proposed solution to accommodate larger solar energy systems and ensure seamless integration with diverse setups.

5.2 Overall Success and Relevance

It is determined that the project was successful in meeting the demand for an advanced solar power measurement system and offering a complete solution for controlling and monitoring solar panel parameters. The project's significance stems from its capacity to enhance the overall objective of sustainable energy solutions through the optimization of solar energy use.

With its emphasis on precise measurements, real-time monitoring, data logging, and adaptive algorithms, the suggested solution has been successfully implemented and is now positioned as a useful instrument in the renewable energy industry. The project's ability to change and adapt to new developments in technology and energy management techniques, in addition to its immediate effects, is what determines how successful it is.

REFERENCES

1. A. A. Felix, O. Hope, O. Joseph and N. Chukwuebuka, "Design and Implementation of a Solar Energy Measurement and Monitoring System," IEEE, Cape Town, 2018.
2. R. Mukaro and X. Carelse, "A microcontroller-based data acquisition system for solar radiation and environmental monitoring," IEEE, 1999.
3. "Solar radiation measurement" written by Tom Stoffel & Steve Wilcox at Aug. 20004.
4. "How a solar cell converts solar Energy to Electricity" written by Turner, Gillian in 2006.
5. microcontrollerslab.com
6. N. Femia, G. Petrone, G. Spagnuolo, and M. Vitelli, "Optimization of perturb and observe maximum power point tracking method," IEEE Trans. Power Electron., vol. 20, no. 4, pp. 963–973, Jul. 2005.
7. N. Kasa, T. Iida, and L. Chen, "Flyback inverter controlled by sensorless current MPPT for photovoltaic power system," IEEE Trans. Ind. Electron., vol. 52, no. 4, pp. 1145–1152, Aug. 2005.
8. H. Shao, C. Y. Tsui, and W. H. Ki, "The design of a micro power management system for applications using photovoltaic cells with the maximum output power control," IEEE Trans. Very Large Scale Integr.

APPENDIX

Code for PIC 16F877A microcontroller

// PIC16F877A Configuration Bit Settings

```
#pragma config FOSC = HS      // Oscillator Selection bits (HS oscillator)
#pragma config WDTE = OFF     // Watchdog Timer Enable bit (WDT disabled)
#pragma config PWRTE = OFF    // Power-up Timer Enable bit (PWRT disabled)
#pragma config BOREN = ON     // Brown-out Reset Enable bit (BOR enabled)
#pragma config LVP = OFF      // Low-Voltage (single-supply) In-Circuit Serial Programming Enable bit (RB3 is digital I/O, HV on MCLR must be used for programming)
#pragma config CPD = OFF      // Data EEPROM Memory Code Protection bit (Code protection off)
#pragma config WRT = OFF      // Flash Program Memory Write Enable bits (Write protection off)
#pragma config CP = OFF       // Flash Program Memory Code Protection bit (Code protection off)
```

```
#include <xc.h>
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```

#define _XTAL_FREQ 20000000 // Define the system clock frequency (20 MHz)

// Define LCD pin connections
#define RS RD2
#define EN RD3
#define D4 RD4
#define D5 RD5
#define D6 RD6
#define D7 RD7

#define buz RD1
#define led RD0

#define EEPROM_ADDRESS 0x00 // EEPROM address for storing the max voltage

void Lcd_Port(char a) {
    if(a
    & 1) D4 = 1; else D4 = 0;
    if(a
    & 2) D5 = 1; else D5 = 0;
    if(a
    & 4) D6 = 1; else D6 = 0;
    if(a
    & 8) D7 = 1; else D7 = 0;
}

void Lcd_Cmd(char a) {
    RS = 0;          // RS = 0 for command mode
    Lcd_Port(a);
    EN = 1;          // Enable pulse
    __delay_ms(4);
    EN = 0;
}

void Lcd_Clear() {
    Lcd_Cmd(0);
    Lcd_Cmd(1);
}

void Lcd_Set_Cursor(char a, char b) {
    char temp, z, y;
    if(a == 1) {
        temp = 0x80 + b - 1;
        z = temp >> 4;
        y = temp & 0x0F;
        Lcd_Cmd(z);
        Lcd_Cmd(y);
    }
    else if(a == 2) {
        temp = 0xC0 + b - 1;
        z = temp >> 4;
        y =

```

```

temp & 0x0F;
Lcd_Cmd(z);
    Lcd_Cmd(y);
}
}

```

```

void Lcd_Init() {
Lcd_Port(0x00);
    __delay_ms(20);

    Lcd_Cmd(0x03);
    __delay_ms(5);
    Lcd_Cmd(0x03);
    __delay_ms(11);
    Lcd_Cmd(0x03);
    Lcd_Cmd(0x02);

```

```

    Lcd_Cmd(0x02);
    Lcd_Cmd(0x08);
    Lcd_Cmd(0x00);
    Lcd_Cmd(0x0C);
    Lcd_Cmd(0x00);
    Lcd_Cmd(0x06);
}

```

```

void Lcd_Write_Char(char a) {
char temp, y;    temp = a &
0x0F;    y = a & 0xF0;
    RS = 1;        // RS = 1 for data mode

    Lcd_Port(y >> 4); // Send higher nibble
    EN = 1;
    __delay_us(40);
    EN = 0;

    Lcd_Port(temp); // Send lower nibble
    EN = 1;
    __delay_us(40);
    EN = 0;
}

```

```

void Lcd_Write_String(char *a) {
    int i;    for(i = 0; a[i] !=
'^0'; i++)
        Lcd_Write_Char(a[i]);
}

```

```

void Lcd_Shift_Right() {
    Lcd_Cmd(0x01);
    Lcd_Cmd(0x0C);
}

void Lcd_Shift_Left() {
    Lcd_Cmd(0x01);
    Lcd_Cmd(0x08);
}

void FloatToStr(float num, char* str, int decimal_places) {
    int integer_part = (int) num;    float fraction_part = num -
    (float) integer_part;    int i = 0;

    // Convert integer part to string
    if (integer_part == 0) {
        str[i++] = '0';    } else {        char
        temp[10];        int idx = 0;        while
        (integer_part > 0) {            temp[idx++] =
        (integer_part % 10) + '0';            integer_part
        /= 10;
        }        for (int j = idx - 1; j >=
        0; j--) {            str[i++] = temp[j];
        }
    }

    str[i++] = '.'; // Add decimal point

    // Convert fractional part to string    for
    (int k = 0; k < decimal_places; k++) {
        fraction_part *= 10;        int digit = (int)
        fraction_part;        str[i++] = digit + '0';
        fraction_part -= digit;
    }

    str[i] = '\0'; // Null-terminate the string
}

void Display_Float(float num, int decimal_places) {
    char buffer[20]; // buffer to hold the string representation of the number
    FloatToStr(num, buffer, decimal_places); // convert the float to a string
    Lcd_Write_String(buffer); // display the string on the LCD
}

```

```

void ADC_Init() {
    ADCON0 = 0x81; // Turn on the ADC module and select channel 1 (AN1)
    ADCON1 = 0x80; // Configure the result format and clock source
}

```

```

unsigned int ADC_Read(unsigned char channel) {
    ADCON0 &= 0xC5; // Clear the channel selection bits
    ADCON0 |= (channel << 3); // Select the A/D channel

    __delay_ms(2); // Acquisition time to charge the hold capacitor

    GO_nDONE = 1; // Start A/D conversion
    while(GO_nDONE); // Wait for the conversion to complete

    return ((ADRESH << 8) + ADRESL); // Return the result
}

```

```

void EEPROM_Write(unsigned char address, unsigned char data) {
    while(WR); // Wait for any previous write to complete
    EEADR = address; // Load address
    EEDATA = data; // Load data
    EECON1bits.EEPGD = 0; // Access data EEPROM memory
    EECON1bits.WREN = 1; // Enable writes
    // Required sequence for write to internal EEPROM
    EECON2 = 0x55;
    EECON2 = 0xAA;
    EECON1bits.WR = 1; // Begin write
    EECON1bits.WREN = 0; // Disable writes
}

```

```

unsigned char EEPROM_Read(unsigned char address) {
    EEADR = address; // Load address
    EECON1bits.EEPGD = 0; // Access data EEPROM memory
    EECON1bits.RD = 1; // Begin read    return EEDATA; //
    Return data
}

```

```

int w; void UART_Init(void); void
UART_Write(char data); void
UART_Write_Text(char* text); void
GSM_Init(void); void GSM_SendSMS(char*
number, char* message); void main() {    w=0;
unsigned int adc_value_voltage, adc_value_current;

```



```

float voltage, current, max_voltage = 0.0;  float
reference_voltage = 5.0; // Reference voltage for
ADC  float sensitivity = 0.185; // Sensitivity of
ACS712 (185 mV/A)
  TRISA = 0xFF; // Set PORTA as input

  TRISD = 0x00; // Set PORTD as output

  ADC_Init(); // Initialize the ADC
  Lcd_Init(); // Initialize the LCD
  buz=1;  led=1;
  __delay_ms(100);
  buz=0;  led=0;
  __delay_ms(200);
  buz=1;  led=1;
  __delay_ms(100);
  buz=0;  led=0;
  Lcd_Clear();
  Lcd_Set_Cursor(1, 1);

  Lcd_Write_String(" Solar Energy");

  Lcd_Set_Cursor(2, 1);
  Lcd_Write_String("Measurment System");
  __delay_ms(2000);
  Lcd_Clear();
  Lcd_Set_Cursor(1, 1);
  Lcd_Write_String(" Embedded ");
  Lcd_Set_Cursor(2, 1);
  Lcd_Write_String(" Systems");
  __delay_ms(1000);
  Lcd_Clear();
  Lcd_Set_Cursor(1, 1);
  Lcd_Write_String(" 22UG10387");
  Lcd_Set_Cursor(2, 1);
  Lcd_Write_String(" 22UG10883");
  __delay_ms(1000);


max_voltage = (float)EEPROM_Read(EEPROM_ADDRESS) / 10.0; // Read max voltage from EEPROM


  while(1) {      adc_value_voltage = ADC_Read(1); // Read the analog value from channel 1 (AN1)
for voltage      voltage = (adc_value_voltage * reference_voltage) / 1023.0; // Convert the ADC value
to a voltage

    // Calculate the actual voltage from the voltage divider

    // Using the voltage divider formula  $V_{out} = V_{in} * (R_2 / (R_1 + R_2))$ 
float R1 = 30000.0; // 30 kOhm      float R2 = 7500.0; // 7.5 kOhm
float Vin = voltage * (R1 + R2) / R2;

    if (Vin > max_voltage) {
max_voltage = Vin;

```

```

EEPROM_Write(EEPROM_ADDRESS, (unsigned char)(max_voltage * 10)); // Store max voltage in
EEPROM

}

adc_value_current = ADC_Read(0); // Read the analog value from channel 0 (AN0) for current
// Convert the ADC value to current      float current_voltage = (adc_value_current * reference_voltage) /
1023.0; // Read the voltage at the output of the ACS712
current = (current_voltage - (reference_voltage / 2)) / sensitivity; // Calculate the current

if(Vin < 3.70){
  Lcd_Clear();
  Lcd_Set_Cursor(2, 1);
  Lcd_Write_String("Low voltage erro");
  Lcd_Set_Cursor(1, 1);
  Lcd_Write_String(" Voltage < 3.7");
  __delay_ms(1000);
  if(w==0){      buz=1;
    __delay_ms(1000);
  buz=0;      UART_Init();
    GSM_Init();
    __delay_ms(5000); // Wait for the GSM module to initialize
    GSM_SendSMS("0711099800", "The solar Voltage is less than 3.7V");//The solar Voltage is less than
3.7V

  w=1;
  }

  }

  else if (Vin > 4.7){
w=0;
  }

  led=1;
  __delay_ms(50);
led=0;
  Lcd_Clear(); // Clear the LCD
  Lcd_Set_Cursor(1, 1);
  Lcd_Write_String("Voltage : ");
  Lcd_Set_Cursor(1, 12); // Set cursor to the first row, first column
  Display_Float(Vin, 2); // Display the voltage on the LCD with 2 decimal places
  Lcd_Set_Cursor(1, 16);
  Lcd_Write_String("V");
  Lcd_Set_Cursor(2, 1);
  Lcd_Write_String("Current : ");

```

```

    Lcd_Set_Cursor(2, 11); // Set cursor to the second row, first column
    Display_Float((current*1000), 2); // Display the current on the LCD with 2 decimal places
    Lcd_Set_Cursor(2, 15);
    Lcd_Write_String("mA");
    __delay_ms(1000);    led=1;
    __delay_ms(50);
    led=0;    Lcd_Clear();
    Lcd_Set_Cursor(1, 1);

    Lcd_Write_String("Power : ");
    Lcd_Set_Cursor(1, 11); // Set cursor to the first row, first column
    Display_Float(Vin*current, 2); // Display the voltage on the LCD with 2 decimal places
    Lcd_Set_Cursor(1, 16);
    Lcd_Write_String("W");
    Lcd_Set_Cursor(2, 1);

    Lcd_Write_String("-----");
    __delay_ms(1000);    led=1;
    __delay_ms(50);
    led=0;    Lcd_Clear();
    Lcd_Set_Cursor(1, 1);

    Lcd_Write_String("Max Voltage:");
    Lcd_Set_Cursor(2, 1);
    Display_Float(max_voltage, 2);
    __delay_ms(1000);
    Lcd_Set_Cursor(1, 1);
    Lcd_Write_String(" Solar Energy");
    Lcd_Set_Cursor(2, 1);
    Lcd_Write_String("Measurment System");
    __delay_ms(1000);

}
}

```

```

void UART_Init(void) {    SPBRG = 31; //
Baud rate 9600 for 20MHz
    BRGH = 1; // High speed
    SYNC = 0; // Asynchronous
    SPEN = 1; // Enable serial port
    TXEN = 1; // Enable transmission
    CREN = 1; // Enable continuous reception
    TX9 = 0; // 8-bit transmission
    RX9 = 0; // 8-bit reception
}

```

```

void UART_Write(char data) {
while(!TRMT);
    TXREG = data;
}

void UART_Write_Text(char* text) {
    int i;
    for(i = 0; text[i] != '\0'; i++) {
        UART_Write(text[i]);
    }
}

void GSM_Init(void) {
UART_Write_Text("AT\r");
    __delay_ms(1000);
    UART_Write_Text("AT+CMGF=1\r"); // Set SMS mode to text
    __delay_ms(1000);
}

void GSM_SendSMS(char* number, char* message) {
    UART_Write_Text("AT+CMGS=\");
    UART_Write_Text(number);
    UART_Write_Text("\r");
    Lcd_Clear();
    Lcd_Set_Cursor(1,1);
    Lcd_Write_String("Sending SMS.....");
    __delay_ms(1000);
    UART_Write_Text(message);
    __delay_ms(1000);
    UART_Write(0x1A); // ASCII code of CTRL+Z to send SMS
    Lcd_Clear();
    Lcd_Set_Cursor(1,1);
    Lcd_Write_String("Sending SMS.....");
    Lcd_Set_Cursor(2,7);
    Lcd_Write_String("Done");
    __delay_ms(1000);
}

```